

Tool and data

Tool and data - paultendo, GitHub.

- Published: date not stated
- Original: <<https://github.com/paultendo/confusable-vision>>
- Preserved from: <https://github.com/paultendo/confusable-vision> (github-repository-api) on 2026-10-02
- Repository commit: e5b74b38d9882c197be942369ffff8c16348df7
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from documentation in [paultendo/confusable-vision](#), pinned to commit [e5b74b38d988](#). GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

README.md

[View original document](#)

confusable-vision

The world's first font-by-font confusables dataset. Which Unicode characters look like which, measured from the fonts' outlines rather than from pixels. The current release, 2026.09.26, compares every letter and digit that each of 322 fonts draws, at the size and baseline position each glyph has in running text. The lookalikes of ASCII letters and digits are then checked in place: set between other letters in common fonts at the size people read them, and compared with pairs everyone accepts as alike, such as 0 and O.

[addons.mozilla.org](#) uses characters from confusable-vision to check add-on names for lookalikes, [credited in Mozilla's source](#); see Used by.

[image: The same rays through the Latin capital O and Ol Chiki letter at (U+1C5B): where each ray enters and leaves ink, and its path through ink, match.]

Used by

- **Mozilla addons-server**, the code behind addons.mozilla.org, adds characters from confusable-vision's output to the table it uses to fold lookalikes to ASCII when checking add-on names: `src/olympia/amo/confusables.py`, pull requests [#24468](#), [#24541](#), [#24612](#) and [#24665](#) (February to March 2026).
- **disarm** ([raeq/disarm](#)), which canonicalises adversarial Unicode before it reaches classifiers, indexes and identifiers, adds measured pairs from confusable-vision to its confusables tables: `data/confusables_supplement.tsv`, `data/confusables_vision.tsv`.
- **SilverSpeak** ([ACMCMC/silverspeak](#)), a Python library for performing and neutralising homoglyph attacks on text, builds the visual neighbours in its homoglyph graph from confusable-vision's discovery files (`docs/hkb.md`).
- **namespace-guard** ships the measured pairs, and through it they reach [agent-sanitizer](#), which uses namespace-guard as its default engine for folding lookalike characters in an AI agent's tool-call input to ASCII ([deps.dev](#)).

These projects took their data from confusable-vision's outputs of February and March 2026.

Discussed on Hacker News: [I rendered 1,418 confusables over 230 fonts](#) and [Confusables.txt and NFKC disagree on 31 characters](#).

Release 2026.09.26 in figures:

- **322 fonts**: every font on macOS, including those it downloads on demand such as PingFang, plus Roboto, Noto Sans, Noto Serif, Noto Sans CJK and DejaVu. **64,751 code points** in 140 scripts.
- **11,517 pairs** alike in at least one font or font combination, of which **5,975** pass the suggested thresholds.
- **719 pairs** of a character and a two-letter ASCII sequence, such as || and ll, or bl and bl.
- **3,055 candidate lookalikes** of ASCII letters, digits and sequences checked in place: 646 alike at the strict tier, 358 more at the broad tier, 101 more that match another common font's design of the letter. 224 of the strict ones are not in Unicode's `confusables.txt`.

The data is published as a versioned release in `data/release/2026.09.26/` (CC-BY-4.0). [namespace-guard](#) and [d0ma1n](#) are built on it.

What changed from release 2

Release 2 (2026.09.24, and 2026.09.25 for the TLD data) compared only the pairs and fonts that earlier runs had compared. An audit of identical glyphs found it caught 637 of the 2,654 cases where a font draws two characters with the same glyph. Release 3 fixes that and adds three things:

- **Coverage**. Every letter and digit each font draws is compared with every other in that font, Han and Hangul included. Release 2 took its characters from a fixed list, so it never reached most of Yi, Canadian Aboriginal, Ethiopic or the Indic scripts, and it missed the script-only fonts and PingFang.

- **The right faces.** Font collections are loaded in the face a browser uses for body text. Earlier runs took the first face, which for some collections was bold, black or italic.
- **Overlapping contours**, as in variable fonts such as San Francisco, are measured as the union of their strokes, as a reader sees them.
- **Two-letter sequences.** Every sequence of two ASCII letters or digits (3,844) is set as each font sets it, with its kerning and ligatures, and compared like a character.
- **The cross-font bar** is set by the upper quartile of the page fonts, so adding fonts to the survey cannot move it.
- **The in-place check** (below).

The March 2026 figures (249,976 single-character pairs and 2,524,275 bigram pairs across 245 fonts) came from measurements that ignored size and baseline. They are superseded; the files from those runs stay in `data/output/` for reference.

How it works

Measuring glyphs

RaySpace casts parallel rays through each glyph's outline at 36 angles, 50 rays each, and records five things per ray: how many times it crosses the outline, where, at what angle, how far it travels through ink each time (its ping, the stroke's width along the ray), and the widest gap between strokes. Each glyph is measured twice: in its own box, for shape, and in a fixed frame on the baseline, so size and position count. Two characters are alike in a font when the shape distance is below 0.5 and the baseline-anchored distance below 0.2. The comparison runs in Rust (`native/cv-pairs`), on every core.

- **Within one font:** every pair of letters and digits the font draws.
- **Across fonts:** a character in its own font, as a browser's fallback would draw it, against the 62 ASCII letters and digits of each of 26 page fonts that lack it. It counts when the ASCII letter is the nearest one, and the character is as close to it as that letter is to itself in the upper quartile of the other page fonts.
- **Sequences:** every character against the 3,844 two-character ASCII sequences, as each font sets them (Core Text; HarfBuzz sets them identically apart from the system font's tracking).

Checking in place

A lookalike measured in isolation can still stand out in a line of text. So every candidate lookalike of an ASCII letter, digit or sequence is set between neighbours (`pa_ne1` , `20_5`) in five contexts, the system font at 13 px and Helvetica, Arial, Times New Roman and Georgia at 16 px, each at a device pixel ratio of 1 and 2, with the platform's font fallback. Nothing is rescaled: the lookalike is compared where it lands in the line.

The bar in each context comes from pairs everyone accepts as alike, 0 and O, 1 and l, and l and I, counted only where that context draws them at the same size and position. A lookalike is:

- **strict** when it is as alike as the median of those pairs and passes every check below;
- **broad** when it is as alike as the least alike of them, with one corner allowed to differ and no slant check;
- **design** when it matches the letter as another common font draws it (a J with a bar at the top, in a line whose J has none).

The checks, on both glyphs rasterised at four times the device resolution: the same number of pieces and holes once gaps narrower than a device pixel close; counters square or round alike, in the same place and of about the same size; spacing to the neighbours; the ink's top, bottom and width; right-to-left characters reordering the digits around them; stroke weight; the shape of each side (mirroring); how far the ink reaches into each corner; slant; a stem in the middle (Y against V); and a bar across a stem (T against T). A two-letter sequence is checked letter by letter.

The tiers were tuned against visual judgements of rendered samples and anchored on the accepted pairs. There has been no study with readers yet.

What it found

Six of the strict lookalikes that are not in Unicode's confusables.txt, each alike in all five contexts. macOS draws each one with a Noto fallback font:

[image: Six lookalikes in words, set in Arial at 46 px and at 16 px beside the real words, with the swapped letter marked.]

Character	Name	Passes for
□ U+1C5B	OL CHIKI LETTER AT	O
□ U+A89D	SAURASHTRA LETTER TTHA	O
□ U+16A60	MRO DIGIT ZERO	O
□ U+11AE4	PAU CIN HAU LETTER FINAL Y	O
□ U+199E	NEW TAI LUE LETTER LOW VA	o
□ U+19D0	NEW TAI LUE DIGIT ZERO	o

The ASCII swaps behind most lookalike domains hold only in some fonts. l for l is near identical in the system font, Helvetica and Arial. 1 for l holds only in Times New Roman, 0 for o only in Georgia, whose digits are lowercase height, and rn for m only in Arial at 16 px on a standard-density screen. d for cl and w for vv do not hold at 16 px.

Along the way: Georgia as shipped with macOS and iOS draws iv (U+2173, SMALL ROMAN NUMERAL FOUR) exactly like ix, nine. Windows 11's Georgia is correct. It was first reported on [Microsoft Q&A](#) in March 2026 and has been reported to Apple.

The release

File	One row per
<code>characters.jsonl.gz</code>	code point measured, with script, category, TR39 identifier type, IDNA2008 status and the TLDs whose registries accept it
<code>lookalikes.jsonl.gz</code>	pair alike in at least one font or combination, with the counts behind it and the distances in each font
<code>sequences.jsonl.gz</code>	character alike to a two-character ASCII sequence
<code>in-place.jsonl.gz</code>	candidate lookalike checked in place, with the verdict and the checks that failed in each context

`DATASET.md` gives every field, what was compared, what an absent pair means and the suggested thresholds. Pin a release by its version. Other committed outputs:

File	Description
<code>data/output/idn-relevant-pairs.json</code>	The IDN view of the release: 2,923 pairs that pass its thresholds and whose characters can both be registered at a common TLD. <code>npx tsx scripts/export-idn-pairs.ts</code>
<code>data/output/confusable-weights-v4.json</code> , <code>font-specific-weights-v4.json</code>	Weights for namespace-guard: pairs involving an ASCII letter or two scripts, scored by the share of fonts where they are alike (overall, and per font). <code>scripts/generate-weights-release.ts</code> , <code>scripts/generate-font-weights-release.ts</code>
<code>data/output/unicode-submission/</code>	Proposed additions to Unicode's confusables data, with the measurements behind each line. <code>scripts/build-unicode-submission.ts</code>
<code>data/input/tld-rules.json</code>	Which characters each delegated TLD's registry accepts at the second level, from IANA's IDN tables, the ICANN registry agreement and researched country-code policies. <code>scripts/build-tld-rules.py</code> ; <code>d0main</code> imports it

Reproducing

Needs macOS (Core Text sets the text), Node 22 or later, Rust, and the Xcode command-line tools for the Swift helpers.

```
npm install
(cd native/cv-pairs && cargo build --release)

# Every pair in every font, across fonts, and the sequences (about two hours on an M-series Mac)
npx tsx scripts/score-all.ts --scope all --cross --sequences --out all-pairs-v8

# The in-place check (about 15 minutes), and the ASCII pairs Unicode lists
node --import tsx scripts/in-place.ts --run all-pairs-v8
node --import tsx scripts/in-place.ts --run all-pairs-v8 --pairs data/input/ascii-known.txt --how "Unicode (ASCII)" \
  --out data/output/all-pairs-v8.ascii.in-place.jsonl

# The release, its IDN view, and the weights
npx tsx scripts/build-release.ts 2026.09.26 --run all-pairs-v8
npx tsx scripts/export-idn-pairs.ts
npx tsx scripts/generate-weights-release.ts data/release/2026.09.26
npx tsx scripts/generate-font-weights-release.ts data/release/2026.09.26

# The README's figures
swiftc -O scripts/render-readme-figures.swift -o /tmp/figs && /tmp/figs docs/images
```

Limits

- The fonts are macOS's, plus Roboto, Noto and DejaVu. Windows fonts are not included.
- The in-place check covers five fonts at 13 and 16 px, at device pixel ratios of 1 and 2.
- Sequences are two characters long.
- The thresholds are measured and anchored on accepted confusables, not tested with readers.

Related

- [namespace-guard](#) ships the measured pairs, weights and in-place lookalikes as runtime data.
- [d0ma1n](#) finds the registered lookalikes of a domain using them.
- [REPORT.md](#): the technical report from the February 2026 SSIM pipeline.

Blog posts

Write-ups on paultendo.github.io. The earlier posts describe earlier runs; the figures above supersede theirs.

RaySpace methodology and findings:

- RaySpace: measuring glyph similarity with vector-outline raycasting
- From CT scanners to confusable characters: the prior art behind RaySpace
- Multi-character confusables: when rn becomes m
- 250,000 confusable pairs. 102 that matter for domain names.

SSIM pipeline findings:

- I rendered 1,418 Unicode confusable pairs across 230 fonts. Most aren't confusable to the eye.
- 793 Unicode characters look like Latin letters but aren't (yet) in confusables.txt
- 28 CJK and Hangul characters look like Latin letters
- 248 cross-script confusable pairs that no standard covers
- 148x faster: rebuilding a Unicode scanning pipeline for cross-script scale
- When shape similarity lies: size-ratio artifacts in confusable detection
- The new DDoS: Unicode confusables can't fool LLMs, but they can 5x your API bill

Background

Posts covering the broader problem space that motivated this project:

- A threat model for Unicode identifier spoofing
- Making Unicode risk measurable
- Your LLM reads Unicode codepoints, not glyphs. That's an attack surface.
- Who does confusable detection actually protect?
- Unicode ships one confusable map. You need two.
- confusables.txt and NFKC disagree on 31 characters

Licence

- **Code** (src/, scripts/, native/, attack-tests/): MIT ([LICENSE](#))
- **Generated data** (data/output/, data/release/): [CC-BY-4.0](#) ([LICENSE-DATA](#)). Free to use, share, and adapt for any purpose including commercial, with attribution.
- **Unicode data** (the .txt files in data/input/): [Unicode License v3](#)
- **Attribution**: Paul Wood FRSA (@paultendo), confusable-vision, <https://github.com/paultendo/confusable-vision>, CC-BY-4.0 (see [NOTICE](#))

`docs/fonts/OFL-googlesanscode.txt`

[View original document](#)

Copyright 2025 The Google Sans Code Project Authors
(github.com/googlefonts/googlesans-code)

This Font Software is licensed under the SIL Open Font License, Version 1.1. This license is copied below, and is also available with a FAQ at: <https://openfontlicense.org>

PREAMBLE The goals of the Open Font License (OFL) are to stimulate worldwide development of collaborative font projects, to support the font creation efforts of academic and linguistic communities, and to provide a free and open framework in which fonts may be shared and improved in partnership with others.

The OFL allows the licensed fonts to be used, studied, modified and redistributed freely as long as they are not sold by themselves. The fonts, including any derivative works, can be bundled, embedded, redistributed and/or sold with any software provided that any reserved names are not used by derivative works. The fonts and derivatives, however, cannot be released under any other type of license. The requirement for fonts to remain under this license does not apply to any document created using the fonts or their derivatives.

DEFINITIONS "Font Software" refers to the set of files released by the Copyright Holder(s) under this license and clearly marked as such. This may include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the copyright statement(s).

"Original Version" refers to the collection of Font Software components as distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting, or substituting -- in part or in whole -- any of the components of the Original Version, by changing formats or by porting the Font Software to a new environment.

"Author" refers to any designer, engineer, programmer, technical writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS Permission is hereby granted, free of charge, to any person obtaining a copy of the Font Software, to use, study, copy, merge, embed, modify, redistribute, and sell modified and unmodified copies of the Font Software, subject to the following conditions:

1. Neither the Font Software nor any of its individual components, in Original or Modified Versions, may be sold by itself.

1. Original or Modified Versions of the Font Software may be bundled, redistributed and/or sold with any software, provided that each copy contains the above copyright notice and this license. These can be included either as stand-alone text files, human-readable headers or in the appropriate machine-readable metadata fields within text or binary files as long as those fields can be easily viewed by the user.

1. No Modified Version of the Font Software may use the Reserved Font Name(s) unless explicit written permission is granted by the corresponding Copyright Holder. This restriction only applies to the primary font name as presented to the users.

1. The name(s) of the Copyright Holder(s) or the Author(s) of the Font

Software shall not be used to promote, endorse or advertise any Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission.

1. The Font Software, modified or unmodified, in part or in whole,

must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software.

TERMINATION This license becomes null and void if any of the above conditions are not met.

DISCLAIMER THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

`docs/fonts/OFL-instrumentsans.txt`

[View original document](#)

Copyright 2022 The Instrument Sans Project Authors
(<https://github.com/Instrument/instrument-sans>)

This Font Software is licensed under the SIL Open Font License, Version 1.1. This license is copied below, and is also available with a FAQ at: <https://scripts.sil.org/OFL>

SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007

PREAMBLE The goals of the Open Font License (OFL) are to stimulate worldwide development of collaborative font projects, to support the font creation efforts of academic and linguistic communities, and to provide a free and open framework in which fonts may be shared and improved in partnership with others.

The OFL allows the licensed fonts to be used, studied, modified and redistributed freely as long as they are not sold by themselves. The fonts, including any derivative works, can be bundled, embedded, redistributed and/or sold with any software provided that any reserved names are not used by derivative works. The fonts and derivatives, however, cannot be released under any other type of license. The requirement for fonts to remain

under this license does not apply to any document created using the fonts or their derivatives.

DEFINITIONS "Font Software" refers to the set of files released by the Copyright Holder(s) under this license and clearly marked as such. This may include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the copyright statement(s).

"Original Version" refers to the collection of Font Software components as distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting, or substituting -- in part or in whole -- any of the components of the Original Version, by changing formats or by porting the Font Software to a new environment.

"Author" refers to any designer, engineer, programmer, technical writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS Permission is hereby granted, free of charge, to any person obtaining a copy of the Font Software, to use, study, copy, merge, embed, modify, redistribute, and sell modified and unmodified copies of the Font Software, subject to the following conditions:

1. Neither the Font Software nor any of its individual components, in Original or Modified Versions, may be sold by itself.

1. Original or Modified Versions of the Font Software may be bundled, redistributed and/or sold with any software, provided that each copy contains the above copyright notice and this license. These can be included either as stand-alone text files, human-readable headers or in the appropriate machine-readable metadata fields within text or binary files as long as those fields can be easily viewed by the user.

1. No Modified Version of the Font Software may use the Reserved Font Name(s) unless explicit written permission is granted by the corresponding Copyright Holder. This restriction only applies to the primary font name as presented to the users.

1. The name(s) of the Copyright Holder(s) or the Author(s) of the Font

Software shall not be used to promote, endorse or advertise any Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission.

1. The Font Software, modified or unmodified, in part or in whole, must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software.

TERMINATION This license becomes null and void if any of the above conditions are not met.

DISCLAIMER THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

`docs/fonts/OFL-syne.txt`

[View original document](#)

Copyright 2017 The Syne Project Authors (<https://gitlab.com/bonjour-monde/fonderie/syne-typeface>)

This Font Software is licensed under the SIL Open Font License, Version 1.1. This license is copied below, and is also available with a FAQ at: <https://scripts.sil.org/OFL>

SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007

PREAMBLE The goals of the Open Font License (OFL) are to stimulate worldwide development of collaborative font projects, to support the font creation efforts of academic and linguistic communities, and to provide a free and open framework in which fonts may be shared and improved in partnership with others.

The OFL allows the licensed fonts to be used, studied, modified and redistributed freely as long as they are not sold by themselves. The fonts, including any derivative works, can be bundled, embedded, redistributed and/or sold with any software provided that any reserved names are not used by derivative works. The fonts and derivatives, however, cannot be released under any other type of license. The requirement for fonts to remain under this license does not apply to any document created using the fonts or their derivatives.

DEFINITIONS "Font Software" refers to the set of files released by the Copyright Holder(s) under this license and clearly marked as such. This may include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the copyright statement(s).

"Original Version" refers to the collection of Font Software components as distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting, or substituting -- in part or in whole -- any of the components of the Original Version, by changing formats or by porting the Font Software to a new environment.

"Author" refers to any designer, engineer, programmer, technical writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS Permission is hereby granted, free of charge, to any person obtaining a copy of the Font Software, to use, study, copy, merge, embed, modify, redistribute, and sell modified and unmodified copies of the Font Software, subject to the following conditions:

1. Neither the Font Software nor any of its individual components, in Original or Modified Versions, may be sold by itself.

1. Original or Modified Versions of the Font Software may be bundled, redistributed and/or sold with any software, provided that each copy contains the above copyright notice and this license. These can be included either as stand-alone text files, human-readable headers or in the appropriate machine-readable metadata fields within text or binary files as long as those fields can be easily viewed by the user.

1. No Modified Version of the Font Software may use the Reserved Font

Name(s) unless explicit written permission is granted by the corresponding Copyright Holder. This restriction only applies to the primary font name as presented to the users.

1. The name(s) of the Copyright Holder(s) or the Author(s) of the Font

Software shall not be used to promote, endorse or advertise any Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission.

1. The Font Software, modified or unmodified, in part or in whole, must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software.

TERMINATION This license becomes null and void if any of the above conditions are not met.

DISCLAIMER THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

`docs/metric-calibration.md`

[View original document](#)

RaySpace metric calibration (release 2)

Release 2026.09.26 keeps this scoring and the thresholds below. It changes what is compared (every letter and digit each of 322 fonts draws, not the pairs earlier runs had found), and adds two-letter sequences and the in-place check. See the README and the release's DATASET.md.

September 2026. How the release 2 scoring differs from the March 2026 run, and the tests behind each change.

Why it changed

Testing the March scores against pairs whose answer is known turned up four problems.

1. **Size and height were invisible.** Rays are spread across each glyph's own bounding box and crossing positions

are fractions of it, so a signature is the same at any size or height. In running text every character sits at one size on one baseline, so o and O, or D and o, look different however alike their shapes. The March scores put ten ASCII case pairs (C/c, V/v, X/x and others) below 0.5.

1. **Scores were averaged only where a pair was found.** A pair's mean distance was taken over the fonts where it

came out alike, never over the fonts that render both characters and show them as different. A pair alike in 12 of 127 fonts (small-capital and all-caps faces) looked as strong as one alike almost everywhere.

1. **Shape differences barely counted.** The distance adds squared differences of crossing positions, angles and

stroke widths (each on a 0 to 1 scale) to a flat 1.0 for every unmatched crossing. Squaring shrinks a real difference: a crossing a quarter of the glyph away adds 0.06. So the distance was mostly a count of crossings. D and O, both rings, scored 0.39 in Arial, where 95% of that came from crossing counts and 0.012 from the stem against the curve; 0 and O, a true lookalike, scored 0.20.

1. **The constants were never checked against known answers.** The weights (0.3 for angles and stroke widths, 1.0 per

unmatched crossing, squared errors, an even split of mean and worst angle) arrived with the scorer on 2 March 2026 with no stated basis. The only validation compared RaySpace with another derived measure (SDF), not with pairs people confuse.

What release 2 does

- **Geometry counts linearly, with weight** (`compareGeometric` in `src/signature-bank.ts`): absolute differences, and

the geometry terms multiplied by 3.

- **A size gate** (`scripts/compute-glyph-boxes.ts` , `scripts/rescore-pairs.ts`): a font only counts when the two

glyphs' tops and bottoms, in em units from the baseline, agree within 0.12 em, and their ink widths within 25% of the wider. Both were calibrated on the labelled pairs. Widths up to 25% and heights up to 0.15 em admitted no false match; at 0.2 em the case pairs (O/o, S/s, V/v, W/w, X/x, whose heights differ by 0.17 to 0.22 em) come in. The width tolerance keeps 0/O (in proportional fonts the zero is 21 to 34% narrower than O), and 0.12 em keeps Thai zero against o, which Thai fonts draw 0.10 to 0.21 em shorter than Latin o. A first version fixed both at 0.06 em without testing, and lost both pairs.

- **The share of fonts**: each pair records how many fonts render both characters and how many show them alike, so a

font-specific quirk stays one. Shares are counted over text fonts: handwriting, display and symbol faces (`src/display-fonts.ts`) are left out, since a pair that only matches in a brush script is not a lookalike where confusables matter.

- **Roboto**, the Latin face on Android and much of the web, is added to the bank from the official Android build

(`scripts/add-font-to-bank.ts` , a side bank beside the system fonts).

- **Across fonts** (`scripts/score-cross-font.ts`): a browser draws a character its page font lacks in a fallback font,

next to Latin in the page font. Within-font scoring cannot see that, and most rare-script characters are only drawn by fonts with no Latin letters at all. Distances across fonts are larger for everything (the same letter in two text fonts has a median distance of 0.98; different letters, 1.82), so the test is relative. Character x in font F looks like target t in reference font R (Roboto and 20 common text faces) when its shape and size gaps to t@R are no larger than the median gaps between t@R and t in the other reference fonts, t is the nearest ASCII letter or digit to x@F, and the distance is at most 0.98. The share is over (F, R) combinations.

Because every term of the new distance is at least as large as the March one, a pair the March run ruled out stays ruled out, and only discovered pairs need re-scoring.

Anchored to the baseline (`normalizeToEmFrame` and a fixed em frame in `computeEnrichedSignature`): the box gate above was replaced by a second signature, with rays across one em frame for every glyph (1.1 em above the baseline to 0.3 em below, the glyph centred on its advance), so size and baseline position are part of the distance. On its own it blurs small features (a dot is only a few rays; i came out like l), so it is used alongside the box signature: a font counts when the box shape distance is below 0.5 and the anchored distance below 0.2. The tolerances on tops, bottoms and widths remain only as the fallback where no anchored signature exists.

Rule (26 text fonts including Roboto)	ASCII kept	ASCII false	Held-out kept	Held-out false
box shape, tops/bottoms within 0.12 em, widths within 25%	4 of 8	0	17 of 23	0
anchored distance only, below 0.2	6 of 8	2 (l/i, i/l)	19 of 23	4 (l/i, n/η, n/π, n/π)
box shape below 0.5 and anchored below 0.2	5 of 8	0	17 of 23	0
box shape below 0.5 and anchored below 0.3	6 of 8	0	17 of 23	1 (l/l)

The labelled sets are small (31 lookalikes), so a difference of one pair is close to noise; the combined rule is at least as good as the tolerances and needs none of them.

`scripts/calibrate-em.ts` reproduces the table.

Viewing size is not modelled yet. At small sizes (a phone address bar) readers miss dots and serifs that these measurements keep, so a pair can be distinct here and still be confused at 11 px. Rasterising each glyph at a given size with the platform's own rendering, and calibrating against published human letter-confusion data, is the next step.

Tests

Distances below 0.5 count as alike; a pair's share is the fraction of fonts rendering both where it is alike at the same size. Twenty-five common text fonts (Arial, Helvetica, Times New Roman, Georgia, Verdana and others).

ASCII letters and digits. Every pair is distinct except TR39's own ASCII mappings (0/O, l/l, 1/l, 1/l, l/l, l/l) and 0/o. This set was used to choose between variants, so it is a development set.

Variant	Lookalikes kept (share ≥ 0.1)	Distinct pairs flagged
March (squared, weight 1)	5 of 8	7, including D/O in 48% of fonts, c/o, O/Q, F/P
absolute, weight 1	5 of 8	3
absolute, weight 3	5 of 8	0

Held out: Latin lowercase against Cyrillic and Greek lowercase. Not used in choosing. TR39's 23 mappings between these are the lookalikes; every other pairing counts as distinct.

Variant	Lookalikes kept (share $\geq$ 0.1)	Distinct pairs flagged
March	20 of 23	10: c/o, c/o, o/c, o/g, n/n (48% of fonts), e/ε, o/α, n/π, x/χ, c/ç
absolute, weight 3	18 of 23	1: x/χ

The two further misses are σ/o and r/r, which only look alike in some fonts. Both variants miss α/a, ι/i and ω/w.

The rules as used (text fonts, Roboto included; alike in at least 3 text fonts, or in all of them when fewer than 3 render both, and at least 5% of the text fonts rendering both). The cap matters for scripts few fonts can draw: Hangul jamo ㅏ against Han ㅏ is identical in the one font that renders both, and would otherwise be ruled out by the count alone. Earlier results with the first rules: on ASCII, l/l and O/o are kept with no false matches; held out, 19 of 23 TR39 lookalikes are kept (missing ι/i, σ/o, r/r, ω/w) and n/n and x/χ are flagged, both arguably real in some fonts.

Across fonts, held out (the same Latin/Cyrillic/Greek set, each character in its own fonts against the reference fonts): with the relative floor alone, 18 of 23 kept and 1 false match at a share of 0.1, but a check of the proposals showed the floor stretching where the reference font is an outlier (a Courier i, a Times E): Bengali ঈ came out like E. With the nearest-letter condition and the cap added, 17 of 23 are kept and there are no false matches.

Limits

- The weight of 3 was chosen from three variants, not fitted. A fitted version needs a larger labelled set and a held-out split.
- Only macOS system fonts and Roboto are measured. Windows fonts and the rest of the Noto family are not yet in the bank, though macOS ships the Noto faces for most rare scripts.
- Letters with a small mark below or above (ạ, ẹ, ọ, ụ) fail the size gate, since the mark extends the glyph's box, yet a reader can miss it. That class is better handled by a rule than by pairs: namespace-guard's `ignoreDiacritics` option removes these marks before comparing.
- Same-script pairs inside the twelve March script sets were not compared; the pair list covers everything registrable against ASCII letters and digits.

Reproducing release 2

The signature bank (`data/output/signature-bank.jsonl.gz` , 7.8 GB) and the March discoveries are not in the repository; set `CV_BANK` to a local copy of the bank if it lives on another drive.

1. `npx tsx scripts/score-pairs-from-bank.ts <pairs.json> data/output/registrable-ascii-discoveries.jsonl`: the registrable characters against ASCII (pair list built from `data/input/tld-rules.json`; the 10-TLD list at release 2026.09.24, extended on 24 Sep 2026 to the 105 characters only other TLDs accept, none of which was found alike).

1. `npx tsx scripts/add-font-to-bank.ts <Roboto-Regular.ttf> Roboto`
`data/output/signature-bank-roboto.jsonl.gz` , then

`score-pairs-from-bank.ts --font Roboto` over Roboto's letters and digits.

1. `npx tsx scripts/compute-glyph-boxes.ts <discoveries>... > data/output/glyph-boxes.jsonl` .

2. `npx tsx scripts/rescore-pairs.ts <discoveries>... > data/output/rescored-pairs.jsonl` .

3. `npx tsx scripts/score-cross-font.ts <pairs.json> data/output/cross-font-pairs.jsonl -`
`-fallback-only` , and

`--validate` for the held-out check.

1. `npx tsx scripts/build-release.ts <version>` , `scripts/export-idn-pairs.ts` and `scripts/build-unicode-submission.ts` .

`docs/score-multichar-optimisation.md`

[View original document](#)

score-multichar.ts optimisation log

Context

`score-multichar.ts` (Milestone 4) scores 3,844 multi-character sequences against 62 single-char targets across ~74 fonts. The inner loop calls `normalisePair()` + `computeSsim()` for every pHash-passing pair -- roughly 3,400-4,400 SSIM computations per sequence, all originally serial.

Before: 1.75 seqs/min, ~36 hours total on a 14-core M-series Mac. **After:** 6.6-7.2 seqs/min, ~8.5 hours total. **~4x speedup.**

What was slow

Each `normalisePair(pngA, pngB)` call ran 7 sharp/libvips operations:

1. `decodeGrey(pngA)` -- sharp pipeline: PNG decode to raw greyscale
2. `decodeGrey(pngB)` -- same for target

3. `sharp(pngA).greyscale().extract(...)` -- crop source from PNG
4. `sharp(pngB).greyscale().extract(...)` -- crop target from PNG
5. Source `applyScale`: `sharp(cropped).resize().extend().png()` -- resize + pad + PNG encode
6. Source raw: `sharp(resized).greyscale().raw()` -- decode PNG back to raw pixels
7. Target `applyScale` + raw -- same two steps for target

With ~4,000 pairs per sequence and all of them serial (`await` in a for loop), the 14-core machine was stuck at 142% CPU (one core + some libvips threading).

Optimisations applied

1. Pre-cache target decode + ink bounds

Targets never change across sequences. `decodeGrey()` + `findInkBounds()` for each target/font combo is computed once at startup and stored in a `Map<string, DecodedGreyWithBounds>` (4,583 entries, ~24s to build).

This eliminates operation #2 from the hot loop entirely.

2. Pre-cache source decode + ink bounds per sequence

Within a single sequence, each source (one per font, ~74 total) is compared against up to 62 targets. The original code called `decodeGrey(src.rawPng)` for every pair -- 62 redundant decodes per source font.

Now source decodes are cached in a per-sequence `Map<string, DecodedGreyWithBounds>` built once before the target loop. This eliminates operation #1 from the hot loop.

3. Pure JS pixel cropping

The original code used `sharp(png).greyscale().extract(...)` to crop -- a full sharp pipeline that re-decodes the PNG. Since we already have raw greyscale pixels from the decode cache, we can crop with a simple row-copy loop:

```
function cropGreyPixels(pixels, srcWidth, left, top, width, height) {
  const out = Buffer.allocUnsafe(width * height);
  for (let y = 0; y < height; y++) {
    pixels.copy(out, y * width, (top + y) * srcWidth + left, (top + y) * srcWidth + left
+ width);
  }
  return out;
}
```

This eliminates operations #3 and #4. Buffer.copy on 48px-wide rows is effectively free compared to a sharp pipeline.

4. Raw-in/raw-out sharp pipeline (skip PNG roundtrip)

The original `applyScale` encoded to PNG, then decoded back to raw pixels for SSIM. Since `computeSsim()` only reads `rawPixels` (never `pngBuffer`), we can feed raw greyscale pixels directly into sharp and output raw:

```
sharp(croppedPixels, { raw: { width: cropW, height: cropH, channels: 1 } })
  .resize(scaledW, scaledH, { fit: 'fill' })
  .extend({ ... })
  .raw()
  .toBuffer();
```

This collapses operations #5+#6 (and #7 for target) from two sharp pipelines each into one. `pngBuffer` is set to `Buffer.alloc(0)` since nothing reads it.

Net result: 7 sharp operations per pair reduced to 2 (one resize+extend+raw per side).

5. Batched concurrency with Promise.all

All pHash-passing pairs for one sequence are collected into a work array, then processed in batches of 12 via `Promise.all`. This lets libvips run multiple resize pipelines concurrently.

```
for (let b = 0; b < work.length; b += CONCURRENCY) {
  const batch = work.slice(b, b + CONCURRENCY);
  const results = await Promise.all(batch.map(async (item) => {
    const [srcNorm, tgtNorm] = await normalisePairCached(item.cachedA, item.cachedB);
    return { ...item, ssimScore: computeSsim(srcNorm, tgtNorm) });
  }));
}
```

CONCURRENCY=12 was chosen to match available cores without overwhelming the thread pool.

6. Pre-built target font lookup maps

Replaced `targets.find(t => t.entry.font === src.entry.font)` (O(74) linear scan, called ~4,600 times per sequence) with a pre-built `Map<string, Map<string, DecodedRender>>` keyed by target character then font name. Minor but free.

7. UV_THREADPOOL_SIZE=16

Node's default libuv thread pool is 4 threads. With 12 concurrent sharp operations, they compete for 4 slots. Setting `UV_THREADPOOL_SIZE=16` as a **shell environment variable** (not `process.env` -- that's too late when tsx's loader has already initialized the pool) lets libvips actually use the available cores.

```
UV_THREADPOOL_SIZE=16 npx tsx scripts/score-multichar.ts
```

What did NOT help

Concurrency alone (v1 attempt)

The first version only cached targets and still called `normalisePairCached(pngA, cachedB, pngBRaw)` which decoded the source PNG fresh every time and used sharp for cropping. Despite 600% CPU utilisation, throughput was *worse* than the original (1.2 seqs/min vs 1.75) due to thread pool contention from too many heavy sharp pipelines competing.

The lesson: reducing the work per item matters more than parallelising expensive items. Going from 7 to 2 sharp ops per pair gave a bigger win than concurrency alone.

`process.env.UV_THREADPOOL_SIZE` (set in JS)

Setting this at the top of the script (before imports) did not reliably work with tsx. The loader initialises the libuv thread pool before the user script runs. Must be set as a shell env var.

What was NOT changed (by design)

- **pHash threshold (0.5):** Raising it loses high-SSIM pairs. Data quality over speed.
- **Worker threads for SSIM:** The greyscale-to-RGBA conversion + `ssim.js` math is fast for 48x48 images (~0.02ms). Message-passing overhead would dominate.
- **Cross-sequence parallelism:** Would break the simple per-sequence progress/resume model.
- **Greyscale SSIM bypass:** Would require importing `ssim.js` internals. Fragile for minimal gain.

Files modified

- `scripts/score-multichar.ts` -- concurrency, font maps, source/target caching, `UV_THREADPOOL_SIZE`
- `src/normalise-image.ts` -- exported `InkBounds`, `DecodedGreyWithBounds`, `decodeAndFindBounds()`, `normalisePairCached()`, `cropGreyPixels()`, `normaliseFromCached()`

Verification

The run is resumable (`progress.jsonl`). SSIM scores should be identical to the serial version since all changes are structural. Spot-check by comparing a few entries from before/after the optimisation boundary in `progress.jsonl`.

